OII462 · PROYECTO DEL MÓDULO

Transformer con puntero para el Weapon-Target Assignment Problem

Un encoder–decoder con atención que aprende a asignar armas a objetivos imitando paso a paso a un algoritmo experto

[Nombre(s) del grupo]

Optimización combinatoria · 2026

Agenda (13–15 min)

1. **Introducción al problema** — 1,5 min
   definición formal y ejemplo visual
2. **Representación del estado** — 2 min
   secuencia de entrada y justificación de cada feature
3. **Generación de datos** — 2 min
   experto, descomposición y codificación de la salida
4. **Arquitectura** — 3 min
   encoder + decoder con puntero
5. **Experimentos y resultados** — 4,5 min
   configuración, convergencia, evaluación, tamaños, conclusiones

Weapon-Target Assignment: definición formal

1 · PROBLEMA

Asignar W armas a T objetivos para minimizar el valor esperado que sobrevive al ataque

ENTRADAS

W armas, T objetivos
$V_j > 0$: valor (amenaza) del objetivo j
$p_{ij} \in [0, 1]$: prob. de que el arma i destruya j ($q_{ij} = 1 - p_{ij}$ sobrevive)

SALIDA

$X = [x_{ij}]$ binaria (W × T): $x_{ij} = 1$ si el arma i va al objetivo j

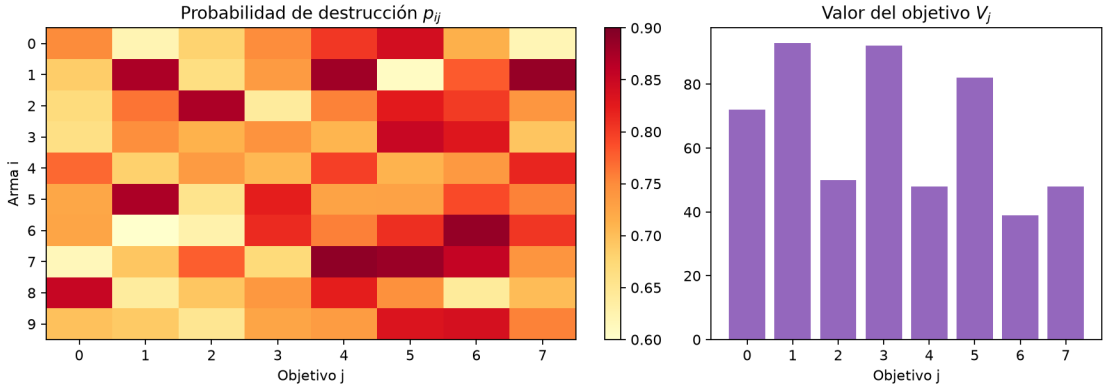
RESTRICCIÓN

$\Sigma_j x_{ij} \leq 1 \quad \forall i$: cada arma a lo más a un objetivo. Un objetivo puede recibir varias armas.

FUNCIÓN OBJETIVO

$$\min_x Z = \sum_{j=1}^{T} V_j \prod_{i=1}^{W} (1 - p_{ij})^{x_{ij}}$$

valor esperado que sobrevive



NP-hard

objetivo no lineal (Lloyd & Witsenhausen, 1986)

$T^W = 8^{10} \approx 10^9$

asignaciones en 10 × 8

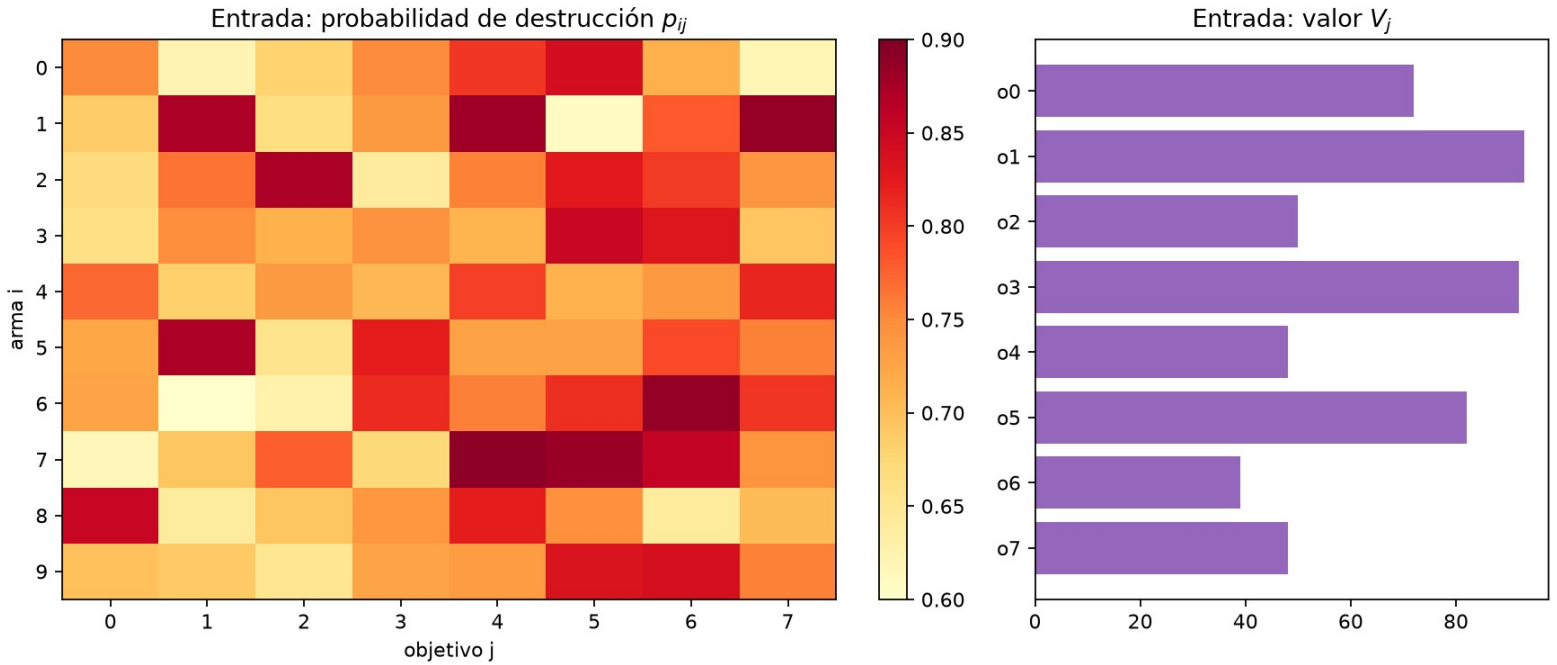
$V_j \sim U\{25..100\}$

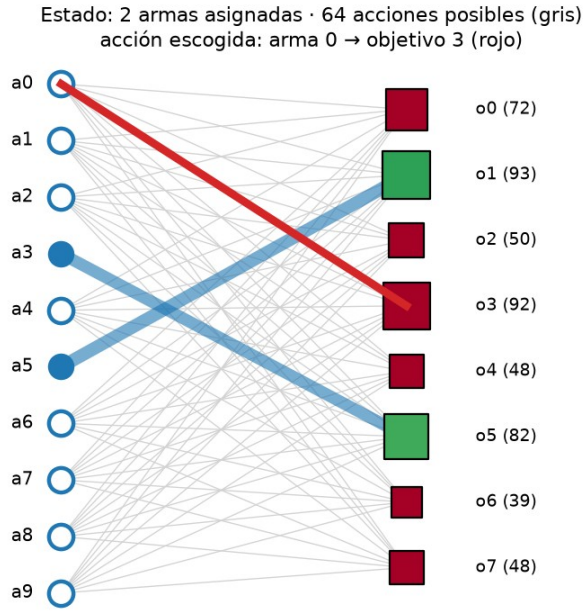
$p_{ij} \sim U(0{,}6; 0{,}9)$ (Ahuja et al., 2007)

Instancia de ejemplo 10 × 8: probabilidades p_{ij} (izq.) y valores V_j (der.)

Un estado y su acción

El modelo construye la solución par a par: en cada paso elige qué arma asignar y a qué objetivo





ESTADO

Asignación parcial: 2 armas ya asignadas (azul). q_j = prob. de que j sobreviva con lo asignado (color del objetivo).

ACCIONES POSIBLES

Todos los pares (arma libre, objetivo): **64 en este paso** (80 al inicio, 8 al final).

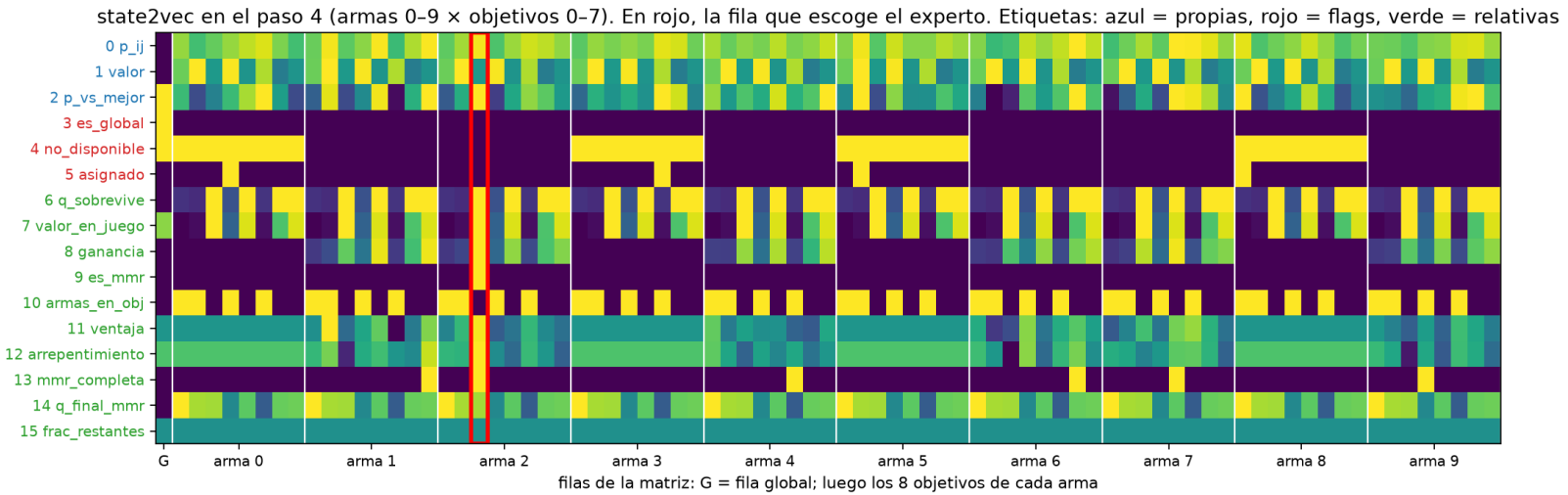
ACCIÓN ESCOGIDA

Arma 0 → objetivo 3 (rojo). Transición: $x_{ij} = 1$ y $q_j \leftarrow q_j \cdot (1 - p_{ij})$.

Secuencia de entrada: una fila por acción posible

2 · REPRESENTACIÓN

Matriz (1 + W·T) × 16 — 81 × 16 en 10 × 8. Todas las filas pasan por los mismos pesos



¿Qué es cada elemento?

fila 0 — **fila global [CLS]**: resume el estado; forma la query

1 + i·T + j — **par (arma i → objetivo j)**: el estado visto desde la acción “asignar i a j”

Con k armas libres hay **k·T filas válidas** = las acciones del entorno.

Propias del par (0–2)

describen la acción por sí misma: efectividad p_{ij} y valor V_j

Flags de estado (3–5)

query (es_global), máscara (no_disponible) y pares ya asignados

Relativas al estado (6–15)

lo que miran las heurísticas: ganancia marginal (MMR), regret y anticipación

¿Por qué pares y no objetivos? MMR, la heurística constructiva clásica del WTAP, evalúa la ganancia de cada par (arma libre, objetivo). Con una fila por par, el puntero elige arma y objetivo a la vez (sin orden fijo de armas), la máscara es natural y el tamaño de la secuencia se adapta a cualquier W y T. Todo está normalizado por instancia: ningún peso depende del índice de la fila.

Features por elemento (0–7): qué capturan y por qué

2 · REPRESENTACIÓN

Azul = propias del par · rojo = flags de estado · verde = relativas al estado (★ = las más importantes)

#	Feature	Qué captura	Por qué es relevante	Relación con la decisión
0	**`p_ij`**	prob. de que el arma i destruya j	define la efectividad de la asignación	pares con p alto reducen más el costo
1	**`valor`**	V_j / max V	el costo pondera cada objetivo por V_j	conviene cubrir antes los objetivos valiosos
2	**`p_vs_mejor`**	$p_{ij} - \max_{j'} p_{ij'}$ (≤ 0)	costo de oportunidad del arma i	0 ⇒ j es el mejor uso posible de esa arma
3	**`es_global`**	1 solo en la fila 0	el decoder debe ubicar la fila que resume el estado	forma la query del puntero
4	**`no_disponible`**	1 si el arma i ya fue asignada (o fila 0)	codifica la restricción $\Sigma_j x_{ij} \leq 1$	máscara: logit -10^9 ⇒ probabilidad 0
5	**`asignado`**	1 si $x_{ij} = 1$ en la solución parcial	contexto: qué pares ya se eligieron	la atención ve qué objetivos ya están cubiertos
6	**`q_sobrevive`**	q_j con lo asignado	rendimientos decrecientes: cada arma actúa sobre lo que queda	q_j bajo ⇒ otra arma en j gana poco
7	**`valor_en_juego`**	$V_j \cdot q_j$ / max V	es el término del costo asociado a j	mucho valor en juego atrae armas

Features por elemento (8–15): qué capturan y por qué

2 · REPRESENTACIÓN

Azul = propias del par · rojo = flags de estado · verde = relativas al estado (★ = las más importantes)

#	Feature	Qué captura	Por qué es relevante	Relación con la decisión
8	**`ganancia ★`**	$V_j\ q_j\ p_{ij}$ / máx. de los pares válidos	reducción inmediata del costo: el criterio de MMR	candidato natural; el modelo aprende cuándo no seguirlo
9	**`es_mmr`**	1 en el par que elegiría MMR	resume la decisión greedy en una columna	referencia explícita que el modelo corrige
10	**`armas_en_obj`**	nº de armas ya en j / W	saturación del objetivo	evita concentrar demasiadas armas
11	**`ventaja`**	p_{ij} − media de p de las armas libres en j	¿es i comparativamente buena para j?	favorece usar el arma “especialista”
12	**`arrepentimiento`**	g_{ij} − mejor g de otra arma libre en j	regret: ¿otra arma lo haría mejor?	negativo ⇒ conviene reservar j para otra arma
13	**`mmr_completa ★`**	1 si al completar con MMR el arma i va a j	anticipa las decisiones futuras	en la Tarea 4, la anticipación fue lo más valioso
14	**`q_final_mmr`**	q_j al completar con MMR	¿quedará j bien cubierto al final?	evita sobre-asignar lo que se cubrirá después
15	**`frac_restantes`**	armas libres / W (en todas las filas)	etapa de la construcción	al inicio hay más margen; al final hay que cerrar

Generación de datos: de la instancia al par (x, y)

3 · DATOS

Aprendizaje por imitación: el experto resuelve, la solución se descompone en decisiones y cada decisión es un ejemplo

Instancia
$V_j \sim U\{25..100\}$
$p_{ij} \sim U(0{,}6; 0{,}9)$
W = 10, T = 8

→

Experto: ILS
MMR + búsqueda local (reassign, swap) + perturbación

→

Solución
10 pares (i, a^*_i) sin orden

→

Descomposición
en cada paso: par del experto con mayor ganancia $V_j q_j p_{ij}$

→

Ejemplo (x, y)
x = state2vec (81 × 16)
y = 1 + i·T + j
(10 por instancia)

¿POR QUÉ ESTE EXPERTO?

Óptimo verificado: 10/10 (10 × 8, meet-in-the-middle) y 30/30 (5 × 4, fuerza bruta).
Rápido: ≈20 ms por instancia; el exacto tarda ≈2 s en 10 × 8 y no escala.
Constructivo vs búsqueda local: imitamos la solución final, no la trayectoria de movimientos (ligada a perturbaciones aleatorias).

DESCOMPOSICIÓN

Estado inicial: ninguna arma asignada.
Acción: par (arma libre, objetivo); k·T posibles.
Orden: mayor ganancia marginal entre los pares del experto pendientes → etiqueta única por estado y reconstruye exactamente su solución.
Término: las W armas asignadas (10 decisiones).

SALIDA (ETIQUETA)

y = índice de la fila escogida (entero) ≡ vector one-hot de largo 1 + W·T.
Las filas enmascaradas nunca son la etiqueta (verificado en todas las muestras).

Pérdida: CrossEntropy sobre los logits enmascarados.

6.000	300	50	+ 120
instancias de entrenamiento	**instancias de validación**	**instancias de prueba (10 × 8)**	**instancias por tamaño**
→ 60.000 pares (x, y)	→ 3.000 pares	las mismas de la Tarea 4	5×4, 15×10, 20×15, 30×20

Arquitectura: encoder + decoder con puntero

El encoder contextualiza la secuencia; el decoder convierte el estado en una distribución sobre las acciones

ENCODER

2 capas · 1 cabeza · d = 64 · se recalcula en cada paso

ENTRA

Secuencia de entrada: (1 + W·T) × 16
fila 0 = global [CLS] · filas 1 + i·T + j = pares (arma i, objetivo j)

HACE

Embedding lineal compartido 16 → 64
mismos pesos para todas las filas

× 2

Self-attention (1 cabeza)
A = softmax(QK^T/√d) · cada par mira a todos
+ residual + LayerNorm

Feed-forward por fila
64 → 128 → 64 (ReLU)
+ residual + LayerNorm

SALE

H ∈ ℝ^((1+W·T) × 64)
un embedding contextualizado por fila: cada par “sabe” del estado global y de los demás pares de la instancia

H

DECODER

1 capa de decisión (sin glimpse) · W pasos

ENTRA

H + flag es_global (col. 3) + máscara (col. 4)
estado parcial ya codificado en H · qué filas son acciones válidas

HACE

Contexto: [h_global ; promedio de h disponibles]
fila [CLS] + embedding del grafo (128) → q = W_c[·] ∈ $\mathbb{R}^{64}$

Puntero (atención de 1 consulta)
u_r = (W_q q) · (W_k h_r) / √64 — un logit por fila

Máscara
u_r = -10^9 si no_disponible = 1 (arma usada o fila global)

SALE

Softmax → P(acción = fila r)
distribución sobre las k·T acciones válidas
GreedyAgent toma el argmax → env.state_transition

Arquitectura en detalle y comparación con Glimpse

75.776 parámetros entrenables · ningún peso depende de W ni de T

Etapa	Operación	Forma (10 × 8)
Entrada	state2vec(estado)	81 × 16
Embedding	Linear(16, 64), compartido	81 × 64
Encoder × 2	self-attention (1 cabeza) + FF 64→128→64, residual + LayerNorm	81 × 64
Contexto	[h_global ; promedio de filas disponibles]	128
Query	q = W_c · contexto	64
Puntero	u_r = (W_q q)·(W_k h_r)/√64, máscara col. 4	81 logits
Salida	softmax(u)	81 probabilidades

POR QUÉ SE RE-CODIFICA EN CADA PASO

Las features incluyen el estado (armas libres, q_j, ganancia, anticipación MMR), así que H cambia en cada decisión. Costo: W pasadas del encoder por instancia (≈**18 ms** por instancia 10 × 8).

Ecuaciones

Encoder (por capa)

$$A = \mathrm{softmax}\left(\frac{QK^\top}{\sqrt{d}}\right), \quad H \leftarrow \mathrm{LN}(H + AV)$$

Decoder: query de contexto

$$q = W_c\,[\,h_{\text{global}}\,;\,\bar{h}_{\text{disp}}\,]$$

Decoder: puntero

$$u_r = \frac{(W_q q)\cdot(W_k h_r)}{\sqrt{d}}, \quad P(r) = \mathrm{softmax}(u)_r$$

VS. GLIMPSE (KOOL ET AL., 2019)

Glimpse: antes del puntero, el contexto atiende con MHA (8 cabezas) sobre H y se refina; los logits se recortan con C·tanh(u).
Nuestro decoder: contexto → puntero directo, 1 cabeza, sin recorte.
Encoder estático vs dinámico: Kool codifica una vez por instancia; nosotros, en cada paso.

5.1 Configuración experimental

5 · EXPERIMENTOS

Todos los hiperparámetros de la arquitectura y del entrenamiento

Arquitectura	Valor
Capas del encoder	2 (self-attention + feed-forward)
Capas del decoder	1 (query de contexto + puntero, sin glimpse)
Cabezas de atención (MHA)	1
Dimensión de los embeddings d	64
Feed-forward	64 → 128 → 64 (ReLU)
Normalización	LayerNorm después de cada residual
Dropout	0,0 (barrido 0 / 0,1 / 0,2 en la Tarea 5)
Parámetros entrenables	75 776

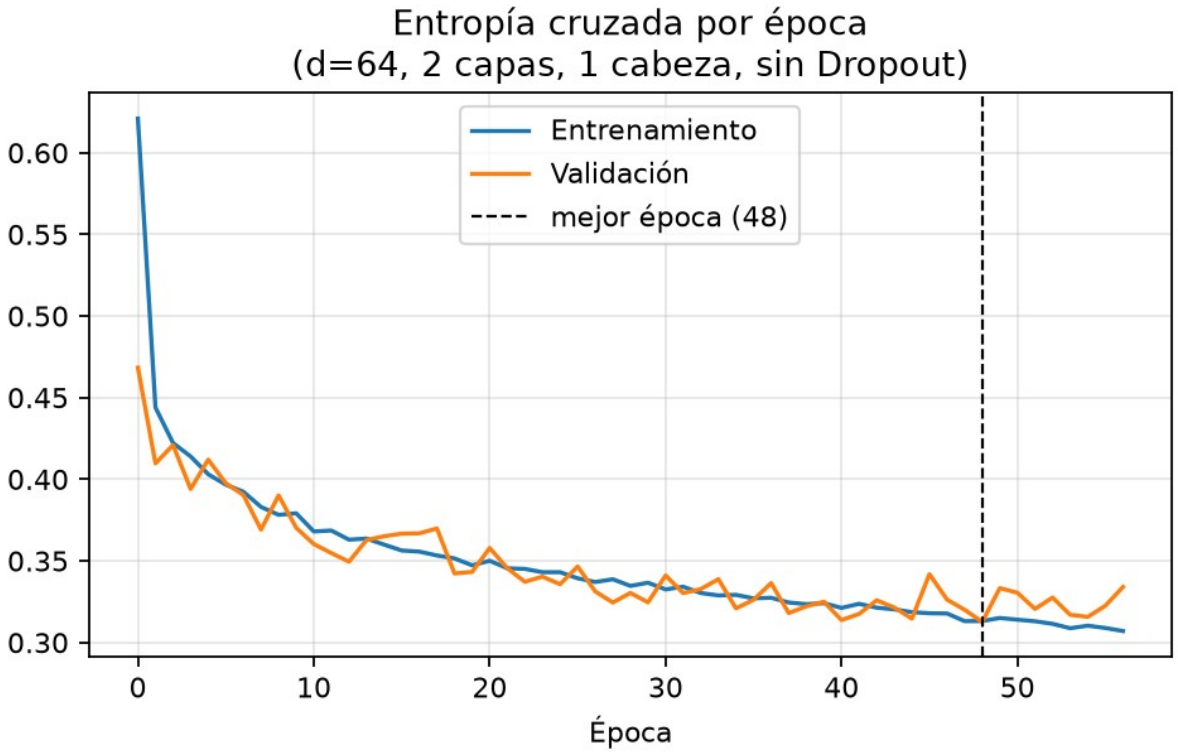
Entrenamiento	Valor
Split entrenamiento / validación	6000 / 300 instancias (60 000 / 3 000 pares estado-acción)
Conjunto de prueba (aparte)	50 instancias 10×8 (las mismas de la Tarea 4)
Épocas	máx. 100; early stopping en la época 56 (mejor: 48)
Batch size	256
Optimizador y tasa de aprendizaje	Adam, lr = 1e-3
Scheduler	no se usó
Early stopping	pérdida de validación, paciencia 8, min_delta 1e-3; restaura la mejor época
Función de pérdida	CrossEntropyLoss sobre logits enmascarados (−1e9 en filas no disponibles)
Métrica de validación	pérdida (early stopping); accuracy y gap del rollout (selección)
Tiempo de entrenamiento	19,3 min en cpu

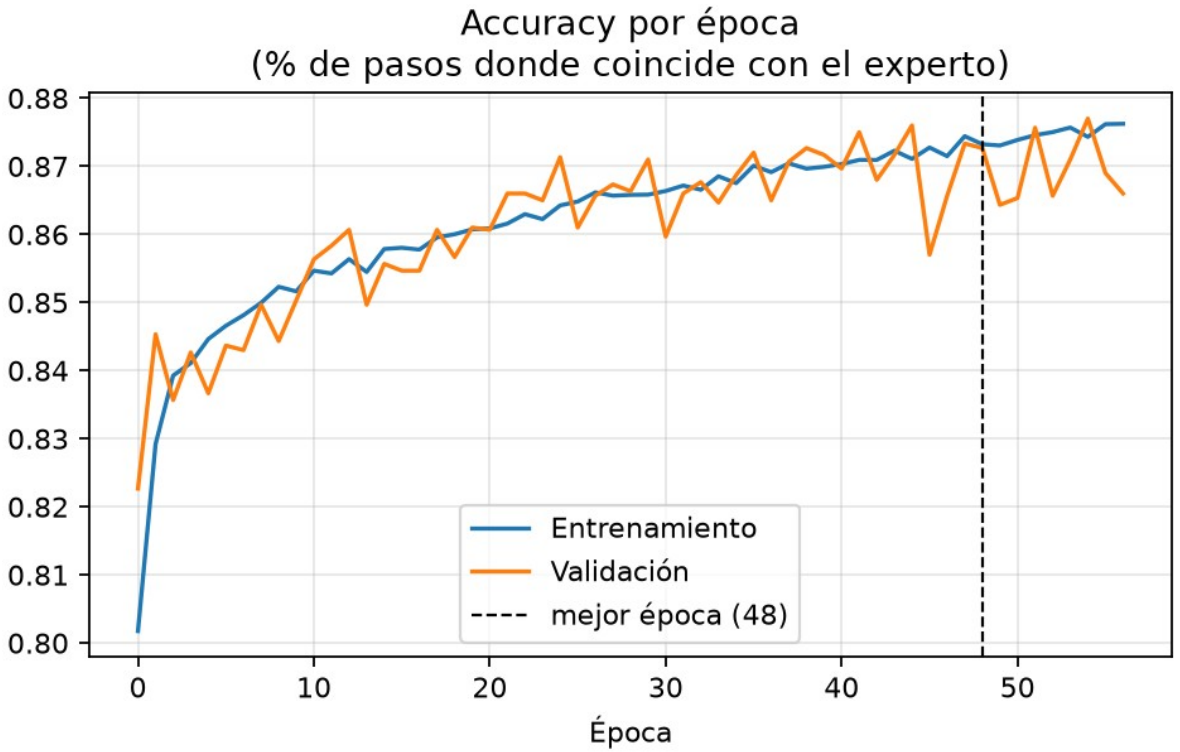
Selección del Dropout (gap rollout en validación)

Dropout 0,0: acc 87,3 % · gap 3,35 %
Dropout 0,1: acc 87,2 % · gap 3,51 %
Dropout 0,2: acc 86,2 % · gap 3,90 %

5.2 Convergencia del entrenamiento

Pérdida y accuracy de entrenamiento y validación por época





época 48

mejor época

de 57 (early stopping)

0,313

pérdida de validación

entrenamiento: 0,313

-0,001

diferencia val − train

≈ 0 ⇒ sin sobreajuste

87,3%

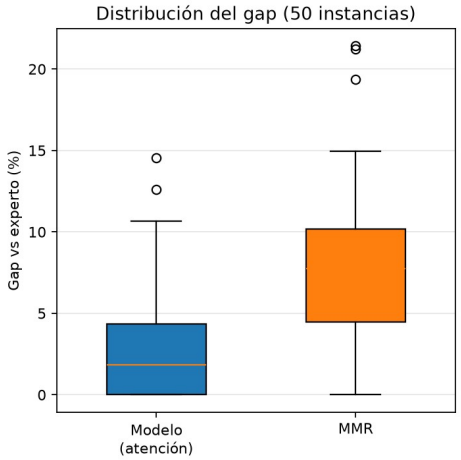
accuracy de validación

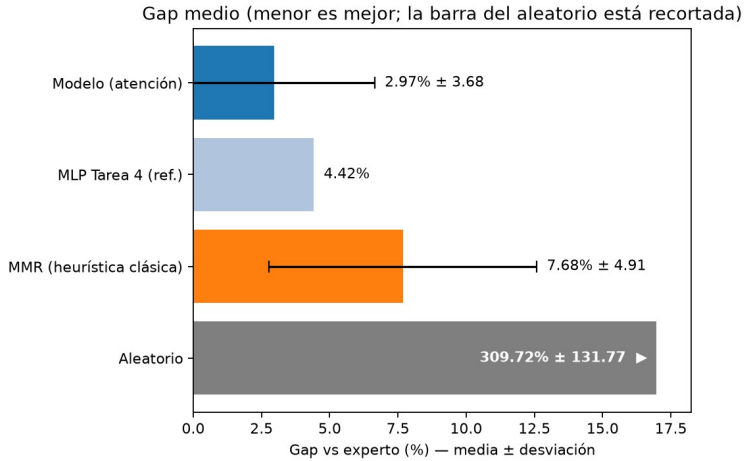
entrenamiento: 87,3%

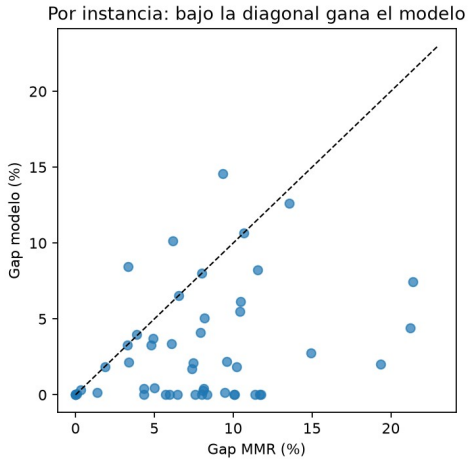
5.3 Evaluación post-entrenamiento

50 instancias nuevas 10 × 8 · gap = 100 · (costo / costo del experto − 1) · el ILS coincide con el óptimo verificado

Método	Costo (media ± desv.)	Gap % (media ± desv.)	Gap mediana	% óptimo	Tiempo / instancia
Modelo (atención)	47,70 ± 8,85	2,97 ± 3,68	1,85%	30%	18,1 ms
MLP Tarea 4 (ref.)	—	4,42 (solo media)	—	—	—
MMR (clásica)	49,83 ± 8,90	7,68 ± 4,91	7,75%	6%	0,1 ms
Aleatorio	186,15 ± 55,39	309,72 ± 131,77	291,64%	0%	≈ 0 ms
Experto (ILS)	46,26 ± 7,98	0,00 ± 0,00	0,00%	100%	20,1 ms





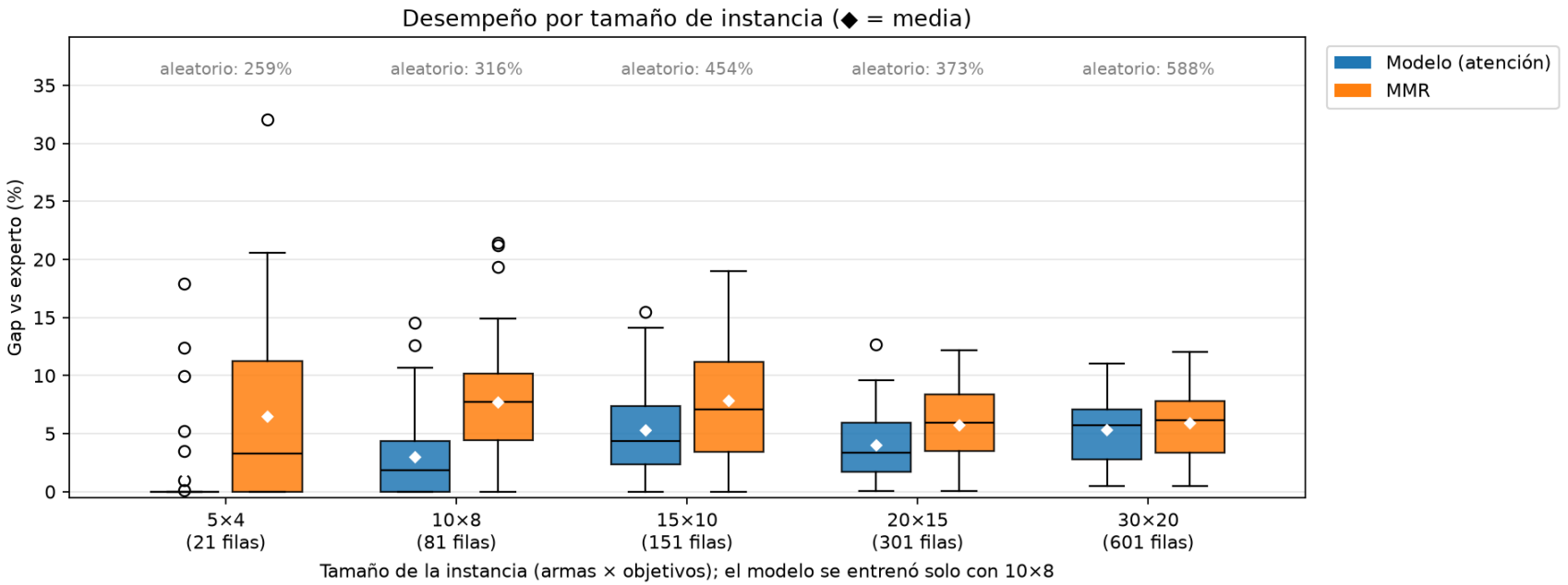


LECTURA

Modelo vs MMR: mejor en 72%, igual en 20% de las instancias.
Gap: 2,97 % vs 7,68 % (MMR) y 4,42 % (MLP T4).
Aleatorio: 310 % ⇒ el modelo sí aprendió.

5.4 Desempeño por tamaño de instancia

Entrenado solo con 10 × 8 (81 filas) y evaluado sin cambios desde 5 × 4 (21 filas) hasta 30 × 20 (601 filas)



W × T	Modelo	MMR	Aleatorio
5×4	1,7 ± 4,2	6,5 ± 7,9	259 ± 102
10×8	3,0 ± 3,7	7,7 ± 4,9	316 ± 100
15×10	5,3 ± 3,9	7,8 ± 4,9	454 ± 160
20×15	4,0 ± 3,1	5,7 ± 3,1	373 ± 90
30×20	5,3 ± 2,9	5,9 ± 3,0	588 ± 104

gap % vs experto (media ± desv.)

¿SE DEGRADA AL ESCALAR?

El modelo supera a MMR en 5 de 5 tamaños. En 10 × 8 (entrenamiento) logra 2,97 %; su mejor tamaño es 5×4 (1,67 %) y el peor 30×20 (5,30 %). La ventaja sobre MMR baja de 4,8 puntos (5×4) a 0,6 en 30×20.

CÓMO LEERLO

Referencia: en 5 × 4 el ILS es óptimo (30/30, fuerza bruta); en tamaños grandes es la mejor solución conocida.
Por qué generaliza: features normalizadas por instancia y pesos compartidos entre filas; solo cambia la longitud de la secuencia.

5.5 Discusión y conclusiones

Qué funcionó, qué no resultó como esperábamos y qué mejoraríamos

¿QUÉ FUNCIONÓ BIEN?

Representación por pares: elige arma y objetivo (sin orden fijo); máscara natural.
Features de las heurísticas: accuracy 87,3% vs 79,9% de la regla MMR.
Calidad: gap 2,97 % vs 7,68 % de MMR, óptimo en 30% de las instancias.
Generaliza de 21 a 601 filas sin reentrenar.

¿QUÉ NO RESULTÓ?

Dropout no ayudó: con 60 mil ejemplos y pesos compartidos no hubo sobreajuste.
Primeras decisiones: accuracy 72% en el paso 0 (80 acciones) vs 100% en el último.
En 30×20 la ventaja sobre MMR cae a 0,6 puntos (gap 5,30 % vs 5,89 %).
Costo: 18 ms por instancia: re-codificar en cada paso y atención $O(n^2)$.

¿QUÉ MEJORARÍAMOS?

Decoder con Glimpse y atención multi-cabeza (Kool et al.).
Entrenar con tamaños mixtos (5 × 4 a 20 × 15) para generalizar mejor.
DAgger o RL para corregir el distribution shift tras un error temprano.
Encoder de W + T elementos con puntero bilineal, para bajar el costo cuadrático.

Conclusión: un encoder–decoder con atención, entrenado solo por imitación, aprende una política constructiva que reduce el gap de MMR de 7,68 % a 2,97 % y que se usa sin cambios en instancias de otros tamaños. La siguiente etapa natural es el glimpse y el entrenamiento por refuerzo.